

## Projektarbetet: LTU – HT26

### Vad ska Ni utveckla?

Projektet ni skall arbeta med under den avslutande modulen är en **läroplattform**, ett så kallat **LMS (Learning Management System)**, anpassat för Lexicons påbyggnadsutbildningar. Ett LMS förenklar och centraliserar kommunikationen mellan lärare, lärosäte och elev genom att samla schema, kursmaterial, övrig information, övningsuppgifter och inlämningar på ett och samma ställe.

Ni skall från grunden producera systemet med databas, back-end funktionalitet och ett genomtänkt frontend. Detta kallas ett ”full-stackprojekt” och syftar till att visa upp er förståelse för samtliga delar av en webbapplikation. Projektet ämnar testa bredden av er förståelse och att ni har en grund att stå på oavsett framtida inriktning.

## Produktbeskrivning

Systemet vi skall bygga har som **främsta uppgift och mål att enkelt tillgängliggöra kursmaterial och schema för elever**. Det skall även fungera som en **samlingsplats för inlämningsuppgifter**.

För att detta skall vara möjligt behöver vi även smidig funktionalitet för lärare att enkelt kunna administrera dessa klasser, elever, scheman och resurser.

För om det inte är enkelt för läraren att använda verktyget, så kommer eleverna aldrig få chansen att använda det.

Det färdiga systemet är ämnat att framför allt täcka grundläggande funktionalitet, men på ett genomtänkt och genomarbetat sätt. "Less is more" är ofta sant när det gäller denna typ av applikationer som skall användas dagligen. Tyvärr, för att nå en så bred marknad som möjligt är de flesta **LMS** som finns tillgängliga idag enormt tunga och överbelamrade av all tänkbar funktionalitet som man sällan har användning för - **detta skall ni ändra på!** "Less is more" behöver inte nödvändigtvis syfta till ren funktionalitet, utan snarare om upplevd komplexitet (**UX**).

**Det får gärna finnas djup funktionalitet, men användaren skall inte behöva fjorton alternativ i varje val den gör (UX).**

## Ramverk och tekniker

Applikationen skall ha en **backend** byggd med **.NET**.

Databasen skall byggas med **Entity Framework Core** enligt **code first**-metoden.

**Frontend** skall använda **React (med eller utan Next.js)**.

Ni får använda er andra ramverk och bibliotek exempelvis **Bootstrap/Tailwind** mm.

## Entiteter, relationer och attribut (grundform)

Nedan beskrivna entiteter och attribut är ett minimum, inte en absolut beskrivning.

Framför allt attributen kommer behöva byggas ut när ni i närmare detalj planerar systemet.

## Användare

**Applikationen skall hantera användare i rollerna av elever och lärare.**

Alla användare skall ha inloggningar och konton i applikationen. Minst namn och e-postadress ska sparas.

## Kurs

**Alla elever tillhör en kurs och endast en.** Kursen har kursnamn, beskrivning och startdatum. Exempel på kursnamn: **“Lexicon LTU”**.

## Modul

Varje kurs läser **en eller flera moduler**. Moduler har modulnamn, beskrivning, startdatum och slutdatum. Exempel på moduler: **“Databasdesign”, “Javascript”** osv. Moduler får inte överlappa varandra eller gå utanför kursens tidsram.

## Aktiviteter

**Modulerna har aktiviteter.** Aktiviteter kan vara e-learningpass, föreläsningar, övningstillfällen, inlämningsuppgifter eller annat.

Aktiviteter har typ, namn, start/sluttid och beskrivning.

Aktiviteter får inte överlappa varandra eller gå utanför modulen.

## Resurs

Alla entiteter ovan kan hålla resurser. En resurs är databaslagrad information kopplad till kurs, modul eller aktivitet.

### Exempel på resurser:

- ✓ Instruktioner
- ✓ Textbaserat kursmaterial
- ✓ Länkar
- ✓ Sammanfattningar
- ✓ Referenser
- ✓ Elevens textinlämningar.

### Resursentiteten skall ha:

- ✓ Namn
- ✓ Beskrivning
- ✓ Tidsstämpel
- ✓ Information om vilken användare som skapade resursen.
- ✓ URL om den pekar på något externt (till exempel kursinformation).

## Use-cases (minimalt krav)

Dessa use-cases är inte heltäckande; beroende på implementation måste mer detaljerade fall tas fram för att täcka in all praktisk funktionalitet.

- ✓ En **icke** inloggad besökare ska kunna
  - Logga in
- ✓ **En elev skall kunna:**
  - Se vilken kurs denne läser och vilka de andra kursdeltagarna är
  - Se vilka moduler denne läser
  - Se aktiviteterna för en specifik modul (modulschema).
- ✓ **En lärare skall kunna:**
  - Se alla kurser
  - Se alla moduler som ingår i en kurs
  - Se alla aktiviteter en modul innehåller
  - Skapa och redigera användare (lärare och elever)
  - Skapa och redigera kurser
  - Skapa och redigera moduler
  - Skapa och redigera aktiviteter

### Use-cases

#### En elev skall kunna:

- ✓ Se om en specifik modul eller aktivitet har resurser kopplade till sig och läsa dessa.
- ✓ Se vilka inlämningsuppgifter denne har fått, om den redan är inlämnad, deadline och om den är försenad.
- ✓ Lämna in textbaserade inlämningar.

#### En lärare skall kunna:

- ✓ Skapa resurser för kurser, moduler och aktiviteter
- ✓ Ta emot textbaserade inlämningar.

### Use-cases (extra om tid finns)

En icke inloggad besökare skall kunna:

- ✓ Begära nytt lösenord

#### En elev skall kunna:

- ✓ Dela resurser med sin kurs eller modul
- ✓ Få notifieringar när en lärare har gjort ändringar i kursen exempelvis lagt till en resurs, modul eller aktivitet.
- ✓ Ta emot feedback på inlämningsuppgifter
- ✓ Få notifieringar när en lärare har lämnat feedback på en inlämningsuppgift

- ✓ Registrera sig själv
- ✓ Ta bort sig från systemet samt radera all information enligt GDPR.

**En lärare skall kunna:**

- ✓ Ge feedback på inlämningsuppgifter

## API (minimalt krav)

- ✓ Ska implementera grundläggande felhantering och returnera lämpliga HTTP-statuskoder.  
Exempel:
  - 404 om resursen inte hittas.
  - 500 för serverfel.
- ✓ Implementerar validering och felhantering.
- ✓ Använda DTOs (Data Transfer Objects) för request/ response
- ✓ Ska vara dokumenterat med hjälp av Swagger, så att tillgängliga endpoints och deras parametrar och responstyper beskrivs.
- ✓ Ska stödja autentisering med JWT.
- ✓ Klienten ska kunna söka/filtrera (valfritt på vad och antal egenskaper)
- ✓ Alla endpoints ska kräva autentisering förutom inloggningen.
- ✓ Klienten ska ha möjlighet att skicka med en pagesize (**frivilligt**)
- ✓ Hantera refreshtokens (**frivilligt**)
- ✓ Stödja paginering (**frivilligt**)

### **React (minimalt krav)**

- ✓ Använda Vite tillsammans med TypeScript och React eller Next.js.
- ✓ Allt ska vara tydligt typat - variabler, argument och returvärden.
- ✓ Genomtänkta och specifika komponenter.
- ✓ Komponenter ska göra en specifik sak.
- ✓ Återanvändbara komponenter skall skapas (buttons osv).
- ✓ Funktioner som inte är reactspecifika, dvs inte behöver använda reactspecifika verktyg ska brytas ut i utils-filer.



## **Frontend**

Krav är att frontend visuellt ska ha ett enhetligt utseende.

Det är tillåtet att använda ramverk som Bootstrap/Tailwind.

Tips för Bootstrap är att titta på de komponenter som finns färdiga för exempelvis forms.

Utöver dessa rent estetiska önskemål skall resterande frontend fokus riktas mot användarupplevelsen och att minska användarens kognitiva friktion.

Systemet ska vara lättanvänt och tydligt.

Applikationen skall vara responsiv.

Bonuspoäng för en välfungerande mobilversion. (Se det som en ren extra uppgift!!!)

## **Arbetssätt**

### **Scrum**

Projektet skall utföras i grupp med ett scrum-baserat arbetssätt.

Vi kommer att arbeta i fem (5) dagars sprintar från måndag till fredag.

En ny sprint startar varje måndag förmiddag med en sprintplanering där ni sätter upp en sprint-backlog, fördelar arbetet och uppdaterar er task-board.

Varje dag inleds med en standup (daily scrum) där ni kort, en och en, avhandlar

1. Vad ni gjort sedan förra standup,
2. Vad ni planerar att göra fram till nästa och
3. Om det är något som blockerar planerat arbete.

Ni håller mötet i Teams appen i gruppens kanal. Ni ska ha er task-board framme så ni visuellt kan se hur ni ligger till. Under fredag eftermiddag avslutar ni sprinten med en sprintdemo, följt av ett retrospektiv.

## **Git - Versionshantering**

Projektet skall versionshanteras med git och GitHub.

- ✓ Minst ha:
  - Feature branch
  - Development
  - Master

## **Test (krav)**

Ni ska använda er av xUnit.

### **1. Domän- och valideringstester (Unit)**

Dessa tester kör snabbt och kräver ingen databas. De säkerställer att era affärsregler håller.

#### **Användare (User)**

- Skapa användare med giltig e-post och namn → ska lyckas.
- Skapa användare med ogiltig e-post → ska kasta valideringsfel.
- Skapa användare utan namn → ska kasta valideringsfel.
- Uppdatera användarroll (elev/lärare) → ska tillåta endast giltiga roller.

#### **Kurs (Course)**

- Skapa kurs med giltiga datum → ska lyckas.
- Kursens startdatum efter slutdatum → valideringsfel.
- Ändra kursnamn → ska uppdatera korrekt.
- Radera kurs med tillhörande moduler → ska hantera Cascade eller blockera enligt design.

#### **Modul (Module)**

- Skapa modul inom kursens tidsram → ska lyckas.
- Modul som överlappar kursens start/slut → valideringsfel.
- Två moduler i samma kurs som överlappar varandra → valideringsfel (om ni har den regeln).
- Uppdatera modulens datum så att den hamnar utanför kursen → valideringsfel.

#### **Aktivitet (Activity)**

- Skapa aktivitet inom modulens tidsram → ska lyckas.
- Aktivitet som överlappar modulens start/slut → valideringsfel.
- Två aktiviteter i samma modul som överlappar → valideringsfel (om ni har den regeln).
- Ändra aktivitetstyp (t.ex. "Föreläsning" → "Inlämning") → ska uppdatera korrekt.

### **Resurs (Resource)**

- Skapa resurs med namn, beskrivning, skapare och valfri URL → ska lyckas.
- Resurs utan namn → valideringsfel.
- Resurs kopplad till kurs/modul/aktivitet → ska spara korrekt relation.
- Uppdatera resursens URL → ska uppdatera korrekt.

### **Inlämning (Submission) – textbaserad**

- Skapa inlämning med text, aktivitet och elev → ska lyckas.
- Inlämning utan text → valideringsfel (om ni kräver innehåll).
- Markera inlämning som “inlämnad” → ska uppdatera status och tidsstämpel.
- Lärare lägger till feedback på inlämning → ska spara korrekt.
- Elev lämnar in efter deadline → status “försenad” sätts korrekt.

**End-to-end:** Helhetsflöden, baserade på use-cases

## **Avstämningpunkter och leverabler**

Under projektets gång förväntas ni redovisa vissa moment innan ni fortsätter.  
Detta för att undvika återvändsgränder och maximera er effektiva utvecklingstid.

- ✓ Produkt-backlog skall godkännas innan ni startar implementation.
- ✓ ER-Diagram skall godkännas innan ni startar en implementation.
- ✓ Wireframes skall uppvisas innan ni startar en implementation.
- ✓ Sprint-backlog skall godkännas innan ni startar en ny sprint.
- ✓ Vid avslutad sprint skall alla leveransklara ändringar demonstreras vid sprintdemo.

## **Planering**

Under projektets uppstart ska ni börja arbeta med att planera arbetet.  
Ni ska ta fram dokumentation enligt punkterna i avstämningpunkter och leverabler.

Fundera på hur systemet ska fungera för lärare och elever. Vad är det primära en lärare är intresserad av? Hur skapas ett bra flöde för att skapa upp nya kurser med allt en sådan består av?

Vad är ni som elever mest intresserade av när ni loggar in?  
Vilken information är mest relevant att få åtkomst till direkt?  
Här handlar det enbart om hur ni presenterar den data som finns lagrad i systemet.  
Hur skall navigation och presentation av data se ut.

## **Redovisning**

Detaljerad information om redovisning momentet kommer senare, men kommer att innehålla use-cases, frågor om kodlösningar, detaljlösningar, felhantering, arbetssätt etc. Systemet skall demonstreras genom en round-trip baserat på några use-cases. Sedan redovisas utvalda delar av kodbasen, några frågor om implementation och arbetssätt besvaras. Detta följs troligen av varma applåder och glada utrop kommentarer och konstruktiv kritik.

Lycka till!